

Machine Learning Notes: Algorithms, Math, and Engineering

Team Notes

January 25, 2026

Contents

1	Introduction to Machine Learning	2
1.1	Definition	2
1.2	Statistics on Big Data	2
1.2.1	Discriminative vs. Generative Models	2
1.3	General Workflow	2
1.4	Three Major Categories	2
2	Model Evaluation Metrics	2
2.1	Regression Problems	2
2.2	Classification Problems	2
2.2.1	Confusion Matrix (Binary)	2
2.2.2	Key Metrics	3
2.2.3	Scikit-Learn Averaging (Multi-class)	3
2.3	Bias vs. Variance (Overfitting vs. Underfitting)	3
3	Linear Models & Optimization	3
3.1	Linear Regression	3
3.2	Derivation of the Normal Equation	3
3.3	Gradient Descent (GD)	4
4	Decision Trees	4
4.1	Tree Construction Algorithms	4
4.2	Impurity Metrics	4
5	Ensemble Learning	4
5.1	Methods	4
5.2	GBDT (Gradient Boosting Decision Tree)	5
5.3	XGBoost (Extreme Gradient Boosting)	5
5.4	LightGBM vs. XGBoost	5
6	Feature Engineering	5
6.1	Steps	5

1 Introduction to Machine Learning

1.1 Definition

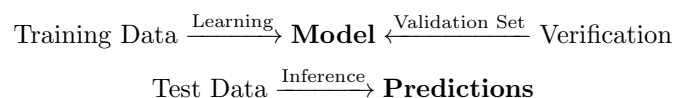
Machine Learning is the algorithm that automatically analyzes data to obtain rules (patterns) and uses these rules to predict unknown data. Given data $(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n)$, the goal is to learn the relationship between X and Y to predict Y_i for a new X_i .

1.2 Statistics on Big Data

1.2.1 Discriminative vs. Generative Models

- **Discriminative Models:** Learn the decision boundary directly (e.g., Logistic Regression, SVM). They model $P(Y|X)$.
- **Generative Models:** Learn the distribution of each class and then determine the boundary (e.g., Naive Bayes). They model $P(X, Y)$ or $P(X|Y)P(Y)$.

1.3 General Workflow



1.4 Three Major Categories

1. **Supervised Learning:** Given (X, Y) , predict Y for new X . (Classification, Regression).
2. **Unsupervised Learning:** Given $X_1, \dots, X_n$, find structure $f(X_i)$. (Dimensionality Reduction, Clustering). The difference is the absence of labels.
3. **Reinforcement Learning** (Not detailed in these notes).

2 Model Evaluation Metrics

2.1 Regression Problems

Given predicted values $\hat{y}_i$ and actual values y_i :

- **MAE (Mean Absolute Error):** $\frac{1}{n} \sum_{i=1}^n |f_i - y_i|$
- **MSE (Mean Squared Error):** $\frac{1}{n} \sum_{i=1}^n (f_i - y_i)^2$
- **RMSE (Root Mean Squared Error):** $\sqrt{\frac{1}{n} \sum_{i=1}^n (f_i - y_i)^2}$
- **MSLE (Mean Squared Log Error):** $\frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2$

2.2 Classification Problems

2.2.1 Confusion Matrix (Binary)

	Guessed Positive	Guessed Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

2.2.2 Key Metrics

- **Accuracy:** $\frac{TP+TN}{All}$
- **Precision:** $\frac{TP}{TP+FP}$ (Focus on correctness of positive predictions)
- **Recall:** $\frac{TP}{TP+FN}$ (Focus on capturing all positive instances)
- **F1-Score:** Harmonic mean of Precision and Recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **F- β Score:** Weighted harmonic mean.

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

Note: If $\beta > 1$, favors Recall. If $\beta < 1$, favors Precision.

2.2.3 Scikit-Learn Averaging (Multi-class)

- **binary:** Only for binary classification.
- **micro:** Calculate metrics globally by counting total TP, FN, FP.
- **macro:** Calculate metrics for each label and find their unweighted mean (commonly used).

2.3 Bias vs. Variance (Overfitting vs. Underfitting)

Metric	Right Fit	Overfitting	Underfitting
Training Error	Low	Low	High
Validation Error	Low	High	High

3 Linear Models & Optimization

3.1 Linear Regression

Goal: Learn $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ such that $f(\mathbf{x}^{(i)}) \approx y^{(i)}$.

Matrix Notation: Let $\hat{\mathbf{w}} = (w_1, \dots, w_d, b)^T$. The prediction for the dataset X is:

$$X = \begin{bmatrix} x_1^{(1)} & \dots & x_d^{(1)} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^{(n)} & \dots & x_d^{(n)} & 1 \end{bmatrix} \Rightarrow \mathbf{y}_{pred} = X \hat{\mathbf{w}}$$

Optimization Objective (Loss Function): Using MSE as loss:

$$\min_{\mathbf{w}} ||X\mathbf{w} - \mathbf{y}||_2^2$$

3.2 Derivation of the Normal Equation

Let $L(\mathbf{w}) = (X\mathbf{w} - \mathbf{y})^T (X\mathbf{w} - \mathbf{y})$. Expanding terms:

$$\begin{aligned} L(\mathbf{w}) &= \mathbf{w}^T X^T X \mathbf{w} - \mathbf{y}^T X \mathbf{w} - \mathbf{w}^T X^T \mathbf{y} + \mathbf{y}^T \mathbf{y} \\ &= \mathbf{w}^T X^T X \mathbf{w} - 2(X^T \mathbf{y})^T \mathbf{w} + \mathbf{y}^T \mathbf{y} \end{aligned}$$

Taking the gradient $\nabla L(\mathbf{w})$ and setting to 0:

$$\nabla L(\mathbf{w}) = 2X^T X \mathbf{w} - 2X^T \mathbf{y} = 0$$

$$X^T X \mathbf{w} = X^T \mathbf{y}$$

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y}$$

3.3 Gradient Descent (GD)

Idea: Iteratively move in the direction of the negative gradient.

1. Choose random start point x_0 , learning rate α , tolerance ϵ .
2. Update rule: $x_{n+1} = x_n - \alpha \nabla f(x_n)$.
3. Stop when $\|\nabla f\| < \epsilon$.

Taylor Expansion perspective:

$$f(x + \delta) \approx f(x) + g^T \delta + \frac{1}{2} \delta^T H \delta$$

Assuming 1st order approximation, we want $g^T \delta < 0$ to decrease the function value. Setting $\delta = -\alpha g$ ensures the steepest descent.

Variations:

- **Batch GD:** Uses all samples (Slow, accurate).
- **Stochastic GD (SGD):** Uses 1 sample (Fast, noisy).
- **Mini-Batch GD:** Uses a batch of samples (Balance).

4 Decision Trees

4.1 Tree Construction Algorithms

The core difference lies in the split criteria and tree structure.

Feature	ID3	C4.5	CART
Split Criteria	Information Gain	Gain Ratio	Gini Index
Features Supported	Discrete	Discrete + Continuous	Discrete + Continuous
Tree Structure	Multi-way	Multi-way	Binary Tree

4.2 Impurity Metrics

1. Information Entropy (Shannon Entropy):

$$\text{Ent}(X) = - \sum_{k=1}^K p_k \log p_k$$

The smaller the entropy, the purer the subset.

2. Gini Index (CART):

$$\text{Gini}(X) = 1 - \sum_{k=1}^K p_k^2$$

For binary classification with probability p :

$$\text{Ent}(p) = -p \log p - (1-p) \log(1-p)$$

$$\text{Gini}(p) = 1 - p^2 - (1-p)^2$$

Gini is computationally cheaper (no logs) and behaves similarly to Entropy.

5 Ensemble Learning

5.1 Methods

- **Bagging (Parallel):** Independent trees trained on bootstrap samples. Example: **Random Forest**.
- **Boosting (Sequential):** Trees are trained sequentially, each correcting the errors of the previous ones. Examples: GBDT, XGBoost, LightGBM, CatBoost.

5.2 GBDT (Gradient Boosting Decision Tree)

GBDT treats the learning process as Gradient Descent in function space.

$$\text{Goal: } \min L(y, f(x))$$

At iteration t , we fit a new tree $f_t(x)$ to the negative gradient (pseudo-residual) of the loss function with respect to the previous prediction.

$$-\nabla_{f(x)} L(y, f_{t-1}(x)) = y - f_{t-1}(x) \quad (\text{for MSE loss})$$

5.3 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized distributed gradient boosting library. Prediction at step K : $\hat{y}_i = \sum_{k=1}^K f_k(x_i)$.

Derivation using 2nd Order Taylor Expansion: Objective at step t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Using Taylor expansion ($f(x + \Delta x) \approx f(x) + f'(x)\Delta x + \frac{1}{2}f''(x)\Delta x^2$):

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n [l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

Where g_i is the 1st derivative (gradient) and h_i is the 2nd derivative (Hessian).

Defining the regularization term $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \|w\|^2$, the optimal weight w_j^* for leaf j is:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

Key Hyperparameters:

- **objective:** Loss function (e.g., 'reg:squarederror').
- **eta:** Learning rate.
- **max_depth:** Max tree depth.
- **num_trees:** Number of boosting rounds.

5.4 LightGBM vs. XGBoost

- **Leaf-wise Growth (LightGBM):** Splits the leaf with max delta loss. Can grow deep (overfitting risk), requires `max_depth` limit.
- **Level-wise Growth (XGBoost):** Splits level by level. Harder to overfit but slower.
- **Histogram-based Learning:** LightGBM bins continuous features into histograms to speed up split finding ($O(\text{data}) \rightarrow O(\text{bins})$).

6 Feature Engineering

6.1 Steps

1. **Exploratory Data Analysis (EDA):** Visualize distributions.
2. **Cleaning:** Handle missing values (Imputation vs Deletion). Note: Tree models can often handle NaNs natively, but Neural Networks cannot.
3. **Feature Selection:** Filter methods (Correlation), Wrapper methods (Recursive elimination), Embedded methods (Lasso/Ridge coefficients).

4. Transformation (Scaling):

- **Standardization (Z-Score):** $Z = \frac{X-\mu}{\sigma}$ (Crucial for NN, SVM, Linear Regression).
- **Min-Max Scaling:** $X' = \frac{X-X_{min}}{X_{max}-X_{min}}$.
- **Log Transform:** $f(x) = \ln(1+x)$ for skewed data (skew > 0.75).

5. Feature Crossing:

- Explicit: Polynomial features, arithmetic combinations ($A \times B$).
- Implicit: Deep Neural Networks (DNN) / DeepFM learn interactions via hidden layers.